Solution Requirements

Date	21 October 2026
Team ID	TDS-AI-2026
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	4 Marks

Project Title

Personalized Travel Itinerary Planner

Functional Requirements

FR-1: User Authentication & Authorization

- Users must be able to sign up with email, password, and display name
- Users must be able to log in with email and password
- Users must be able to log out
- All API endpoints (except health check) must require a valid JWT bearer token
- JWT tokens must be verified using Supabase JWKS (public key rotation supported)
- Automatic profile creation on signup via database trigger

FR-2: Trip Management

- Users must be able to create a new trip with: destination, start date, end date, budget, currency, group size, travel style, interests, and optional notes
- Users must be able to list all their trips (dashboard view)
- Users must be able to view a single trip with full day-by-day itinerary details
- Users must be able to update trip metadata (title, destination, dates, budget, etc.)
- Users must be able to delete a trip (cascading delete of days and activities)
- Trip duration is limited to a maximum of 14 days

FR-3: AI-Powered Itinerary Generation

- The system must generate a complete multi-day itinerary using a Large Language Model (Gemini or Groq)
- Generated itineraries must include day summaries and activities with: type, title, description, location name, start/end times, and estimated cost
- The AI must use real places at the specified destination
- The AI must respect the user's budget, travel style, and interests
- The system must validate AI output against Pydantic schemas before saving
- The system must retry once on invalid AI output

FR-4: Day Regeneration

- Users must be able to regenerate a single day's itinerary without affecting other days
- The regeneration must avoid repeating activities from other days
- Old activities are replaced atomically (insert new, then delete old)

FR-5: Activity Management

- Users must be able to edit any activity (type, title, description, location, times, cost)

- Users must be able to delete individual activities
- Users must be able to reorder activities within a day via drag-and-drop
- Reordering must update position indices and swap time slots

FR-6: Budget Tracking

- The system must compute per-day estimated costs from activity costs
- The system must compute total trip estimated cost from day costs
- Budget information must be displayed on the dashboard and trip detail pages

---

Non-Functional Requirements

NFR-1: Performance

- AI itinerary generation should complete within 10-30 seconds
- API response times for CRUD operations should be under 500ms
- Frontend should load within 3 seconds on standard broadband

NFR-2: Security

- All data access must be governed by Row-Level Security (RLS) in Supabase
- Users can only access their own data (or data shared with them)
- Passwords are hashed by Supabase Auth (never stored in plaintext)
- CORS is configured to only allow the frontend origin
- Request logging includes unique request IDs for traceability

NFR-3: Reliability

- LLM failover: if Gemini fails after 3 retries, automatically fall back to Groq
- Failed trip generation must not leave partial data in the database (atomic transactions)
- Input validation on both frontend (Zod) and backend (Pydantic)

NFR-4: Scalability

- Stateless backend design allows horizontal scaling
- Database connection pooling via Supabase
- Per-request Supabase client scoped to the authenticated user

NFR-5: Usability

- Clean, modern UI with responsive design (mobile and desktop)
- Form validation with real-time error messages
- Loading states and progress indicators during AI generation
- Drag-and-drop support for activity reordering